Bases de données avancées

GLO-4035

Automne 2024

Réalisé par

Zack Dufresne (536 899 030)

Charles-Étienne Dumont (536 960 589)

Charles Brassard (536 783 602)

Christian Fosso (536 904 549)

Dérisquer l’application

Travail présenté à

Eloïse Prévôt

8 novembre 2024

Table des matières

Stratégie d’acquisition des données....4
Source et méthode d’extraction....4
Présentation de la ville....4
Source de données....4
Méthode d’obtention des données....4
Analyse des données....4
Présentation d’exemples de données sources....4
Technologies utilisées....5
Langage de programmation....5
Base de données....5
Les détails du processus d’extraction, de transformation et de conversion (ETL)....6
Processus d’acquisition initiale des données....6
Processus d'Acquisition Incrémental des données....6
Processus de transformation des données....7
Schéma du pipeline d’ETL....7
Annexes....9
Annexe 1....9
Annexe 2....9
Annexe 3....9
Annexe 4....9
Annexe 5....10
Annexe 6....10
Annexe 7....11
Annexe 8....11
Annexe 9....11
Annexe 10....11
Annexe 11....11
Annexe 12....13

Annexe 13.....14
Annexe 14.....15
Annexe 15.....16
Annexe 16.....17
Annexe 17.....17
Annexe 18.....18

Stratégie d’acquisition des données

Source et méthode d’extraction

Présentation de la ville

La ville que nous avons choisie pour réaliser ce travail est Paris. Celle-ci offre un vaste réseau de pistes cyclables bien développées et une multitude de restaurants allant des bistrots traditionnels aux établissements étoilés. C’est la ville idéale pour une balade à vélo épicurienne.

Source de données

Les données géospatiales utilisées pour ce projet seront récupérées de OpenStreetMap (OSM). C’est une base de données open source qui fournit beaucoup d’informations détaillées sur les routes et points d’intérêts à partir d’une localisation. Grâce au langage de programmation utilisé qui sera présenté plus bas, nous pouvons utiliser la bibliothèque “osmnx” pour récupérer les coordonnées des restaurants de la ville. Nous allons par la suite récupérer à partir de OpenRouteService les itinéraires optimaux à vélo entre deux coordonnées géospatiales, qui combiné avec OSM nous permettra d’obtenir toutes les données nécessaires.

Méthode d’obtention des données

Pour obtenir les données sur les restaurants, nous utiliserons du code similaire à celui présenté à l’Annexe 1, en appliquant un champ “amenity” à restaurant et en ajustant la ville à Rimouski. Pour obtenir les données des itinéraires de pistes cyclables de la ville, nous utiliserons une requête vers l’API de OpenRouteService similaire à celle présentée en Annexe 2.

Analyse des données

Les données sur les restaurants récupérées seront sous la forme d’un GeoDataFrame à partir duquel il est très simple d’extraire les données géospatiales requises pour la suite ou même d’effectuer des opérations de calcul sur ces données. Pour les itinéraires à vélo, la requête présentée en Annexe 2 (où la clé API est omise intentionnellement) retourne un objet JSON qui contient toutes les informations d’itinéraires nécessaires.

Présentation d’exemples de données sources

Voir annexes 3 à 11 pour des exemples de données

Technologies utilisées

Langage de programmation

Pour notre travail longitudinal, nous avons décidé d'utiliser Python comme langage de programmation. Ce choix a été adopté notamment pour la simplicité et l'accessibilité de ce dernier qui contribueront à la maintenabilité du système. En ce sens, nous considérons que Python est un outil populaire qui possède une vaste communauté et une documentation abondante, ce qui permettra à tous une meilleure autonomie. Par ailleurs, celui-ci permet d'obtenir du code lisible, clair et intuitif, ce qui facilitera la compréhension et la communication au sein de l'équipe. De plus, Python peut aussi contribuer à la fiabilité du système. D'une part, il offre un moyen robuste de gérer les exceptions grâce à des blocs try-except, ce qui permettra un meilleur contrôle des erreurs. D'une autre part, il permet de détecter les problèmes rapidement et de les corriger au fur et à mesure en effectuant des tests automatisés grâce au framework pytest.

À noter que le langage JavaScript sera utilisé pour implémenter une partie de notre API. Celui-ci permet notamment une bonne réutilisation du code, ce qui contribuera à la maintenabilité du système. Pour en ajouter, il permet une bonne fiabilité pour les mêmes raisons que Python et par sa gestion des erreurs avec "promises".

Base de données

Les types de bases de données que nous avons choisis pour effectuer notre travail longitudinal sont les bases de données orientées document et graphe. Tout d'abord, nous avons choisi le type document car il permet efficacement de stocker les données dans un format qui peut facilement être modifié et qui n'a pas à être constant si les données ne le sont pas elles-mêmes. Ce type est donc bien extensible, ce qui sera un avantage si on doit ajouter de nouvelles données, dont certaines ne seront peut-être pas dans le même format que celles présentes dans la base de données. Dans notre cas, ce type sera utilisé pour stocker les informations sur les restaurants. Nous avons également utilisé le type graphe, car celui-ci permet efficacement de représenter les relations *many-to-many* comme dans notre cas, les réseaux de pistes cyclables. Similairement aux bases de données de types document, celles de types graphes sont extensibles,

ce qui sera utile si on doit ajouter de nouveaux types de relations ou étendre les réseaux de pistes cyclables.

Les détails du processus d’extraction, de transformation et de conversion (ETL)

Processus d’acquisition initiale des données

L’extraction initiale des données a été réalisée de deux manières. Premièrement, nous avons utilisé la base de données géographiques OpenStreetMap (OSM) pour extraire les points d'intérêt correspondant aux restaurants. OSM propose une grande variété d’informations, mais nous avons tamisé ceux-ci en gardant uniquement osmid, name, cuisine, coordinates, addr:city, addr:housenumber, addr:postcode, addr:street, opening_hours, phone et description. Ces dernières suffisent pour effectuer une analyse détaillée qui nous permettra de localiser les restaurants. Deuxièmement, nous avons utilisé la même source pour extraire les données relatives aux pistes cyclables. Pour ce cas de figure, nous avons conservé les coordonnées sous forme de liste de points pour une représentation plus exhaustive des chemins.

Voir l’annexe 12 pour le diagramme UML plus détaillé du processus d’extraction

Processus d'Acquisition Incrémental des données

Dans le cadre de ce projet, un processus d'acquisition incrémentale des données a été défini afin de maintenir les bases de données MongoDB et Neo4J à jour. Cela permet d’éviter les doublons et de gérer efficacement les mises à jour et ajouts futurs. Dans le cas de MongoDB, nous vérifions pour chaque restaurant s’il existe déjà dans la base de données en s’appuyant sur l’ID de ce dernier. Si une correspondance est trouvée, nous mettons à jour les informations nécessaires. Dans le cas contraire, un nouveau document est ajouté à la collection. Dans le cas de Neo4J, pour chaque segment de piste cyclable, nous utilisons des requêtes *MERGE* afin de vérifier l'existence des nœuds de début et de fin, ainsi que de la relation entre eux. Si le segment est nouveau, il est ajouté en tant que nouvelle relation entre les nœuds correspondants.

Voir l’annexe 13 pour le diagramme illustrant le flux des étapes dans le processus d'acquisition incrémental pour MongoDB et Neo4j.

Processus de transformation des données

Pour obtenir des données plus compactes et parlantes, nous avons réalisé quelques transformations simples. Pour commencer, nous avons standardisé les adresses des restaurants en appliquant un format canadien. Ainsi, chaque adresse est désormais représentée sous la forme "Numéro de maison, Rue, Ville, Code postal". Par la suite, nous avons traduit les colonnes en français et renommé certaines d'entre elles pour qu'elles soient plus adaptées à notre contexte. Effectivement, la colonne *name* a été traduite par *nom*, et la colonne cuisine a été remplacée par *type_de_restaurant*. Finalement, nous avons procédé à la classification des restaurants et services. En ce sens, les différents restaurants sont maintenant divisés par type selon la colonne *type_de_restaurant* et on a ajouté la colonne *type_de_service* pour classifier les services. Cette dernière a été créée à partir de la description du restaurant. Par exemple, "emporté" est classé comme "Service à emporter", "uber eats" comme "Service de livraison", "buffet" ou "À volonté" comme "Buffet / À volonté". Cette distinction est essentielle pour mieux comprendre l'expérience de restauration proposée.

Voir l'annexe 14 pour le diagramme illustrant le flux des étapes de transformation.

Schéma du pipeline d'ETL

Les deux types de base de données utilisés dans le cadre de notre application sont bien complémentaires. En effet, nous utilisons MongoDB comme stockage principal de nos restaurants, où le format JSON offert par celui-ci nous permet d'obtenir des données structurées et flexibles pour chacun des restaurants. Neo4j nous permet ensuite d'établir des relations avec les pistes cyclables et les restaurants.

Ainsi, les données transformées des restaurants sont chargées dans MongoDB pour permettre des recherches et des analyses rapides sur les options de restauration. Nous commençons évidemment par une connexion à notre base de données. Par la suite, nous éliminons les doublons avant de convertir le fichier CSV contenant les données de restaurants transformées en liste de dictionnaires Python. Cette dernière étape est prérequise à l'insertion des données dans notre collection MongoDB.

Par ailleurs, les pistes cyclables sont structurées dans Neo4j en tant que réseau de connexions entre différents points géographiques, ce qui est favorable pour des requêtes orientées graphes, comme la recherche de trajets. Après avoir établi la connexion à notre base de données Neo4j, nous procédons à

l’élimination des doublons, puis au chargement des données. Par la suite, nous utilisons le fichier CSV contenant les données sur les pistes cyclables et des commandes *MERGE* pour créer des nœuds basés sur les coordonnées géographiques et leurs relations. Finalement, nous utilisons aussi le même procédé avec le fichier contenant les données des restaurants transformés pour la création des nœuds.

Voir l’annexe 15 pour un schéma du pipeline d’ETL global.

Annexes

Annexe 1

Exemple de code Python pour récupérer des “amenity” restaurant de la ville Rimouski

```python
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place("Rimouski, Quebec, Canada", tags)
```

Annexe 2

Exemple de requête envoyé vers l’API de OpenRouteService pour l’itinéraire en vélo entre deux points (dans notre cas, les points seront des coordonnées de restaurants, la clé est omise par standard de confidentialité)

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=8.681495,49.41461&end=8.687872,49.420318
```

Annexe 3

Exemple de structure d’un GeoDataFrame créé par le code de l’Annexe 1 qui contiendra les restaurants de la ville (généré par intelligence artificielle ChatGPT 4o)

```
   osmid       name              geometry                amenity   addr:street
0  123456789   Chez Gourmet      POINT (-68.52 48.44)    restaurant Rue Principale
1  987654321   Bistro du Port    POINT (-68.53 48.45)    restaurant Avenue de la Mer
```

Annexe 4

Exemple d’objet JSON retourné par une requête de format similaire à celle présentée à l’Annexe 2 (les champs distance (en mètres) et duration (en secondes) correspondent aux détails du trajet global, le champ concaténé “steps” énumère les différentes étapes du trajet avec instruction de déplacement progressive)

```
{
    "type": "FeatureCollection",
    "bbox": [8.681423, 49.414599, 8.690123, 49.420514],
    "features": [
      {
        "bbox": [8.681423, 49.414599, 8.690123, 49.420514],
        "type": "Feature",
        "properties": {
          "segments": [
            {
              "distance": 1408.8,
              "duration": 281.9,
              "steps": [ ···
              ]
            }
          ],
          "way_points": [0, 23],
          "summary": {
            "distance": 1408.8,
            "duration": 281.9
          }
        },
```


Annexe 5

Exemple d’erreur de collecte de données, la ville de Los Angeles n’étant pas au Québec

```
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place("Los Angeles, Quebec, Canada", tags)
```

Annexe 6

Exemple d’erreur de collecte de données en omettant l’emplacement désiré

```
import osmnx as ox

tags = { "amenity": "restaurant", "highway": "cycleway" }

gdf = ox.features_from_place(tags)
```

Annexe 7

Exemple d'erreur de collecte de données, récupérant les banques au lieu des restaurants, en plus d'une faute dans le nom de la ville

```
import osmnx as ox

tags = { "amenity": "bank", "highway": "cycleway" }

gdf = ox.features_from_place("Rimousky, Quebec, Canada", tags)
```

Annexe 8

Exemple d'erreur d'extraction de données, le mode de transport est la voiture, ce qui ne correspond pas à ce que l'on souhaite

```
GET https://api.openrouteservice.org/v2/directions/driving-car?api_key=your-api-key&start=
8.681495,49.41461&end=8.687872,49.420318
```

Annexe 9

Exemple d'erreur d'extraction de données, la clé API est omise et laissé à nulle, ne permettant pas la récupération des données

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=null&start=
8.681495,49.41461&end=8.687872,49.420318
```

Annexe 10

Exemple d'erreur de données, utilisant des coordonnées géospatiales invalides en dehors des limites conventionnellement établies

```
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=
95.000000,-195.000000&end=8.687872,49.420318
```

Annexe 11

Exemple d'erreur de formatage de données, utilisant de virgules "," pour les coordonnées de départ à la place de points "."

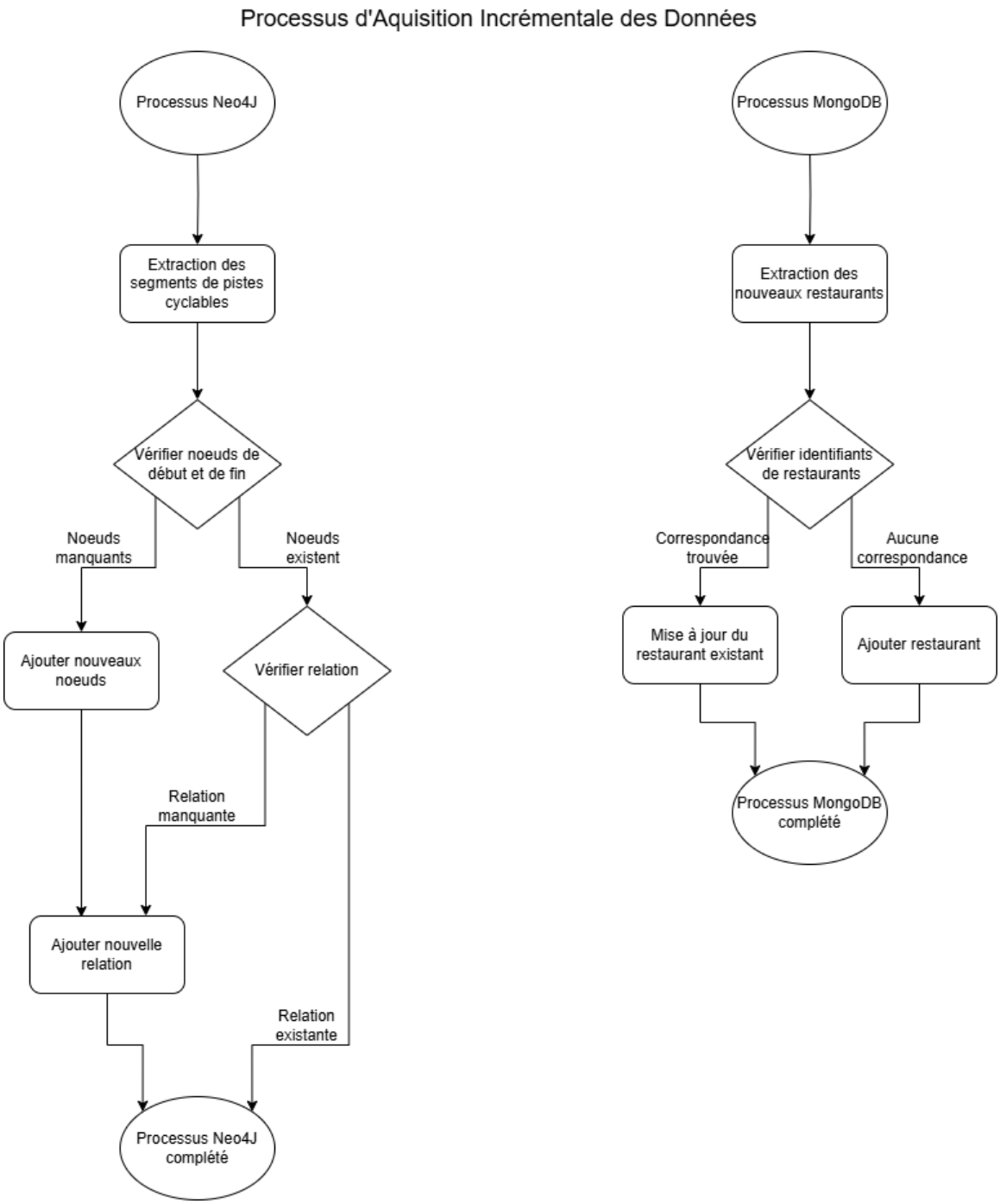
GET https://api.openrouteservice.org/v2/directions/cycling-regular?api_key=api-key&start=8,681495,49,41461&end=8.687872,49.420318

Annexe 12

Diagramme UML du processus d’extraction

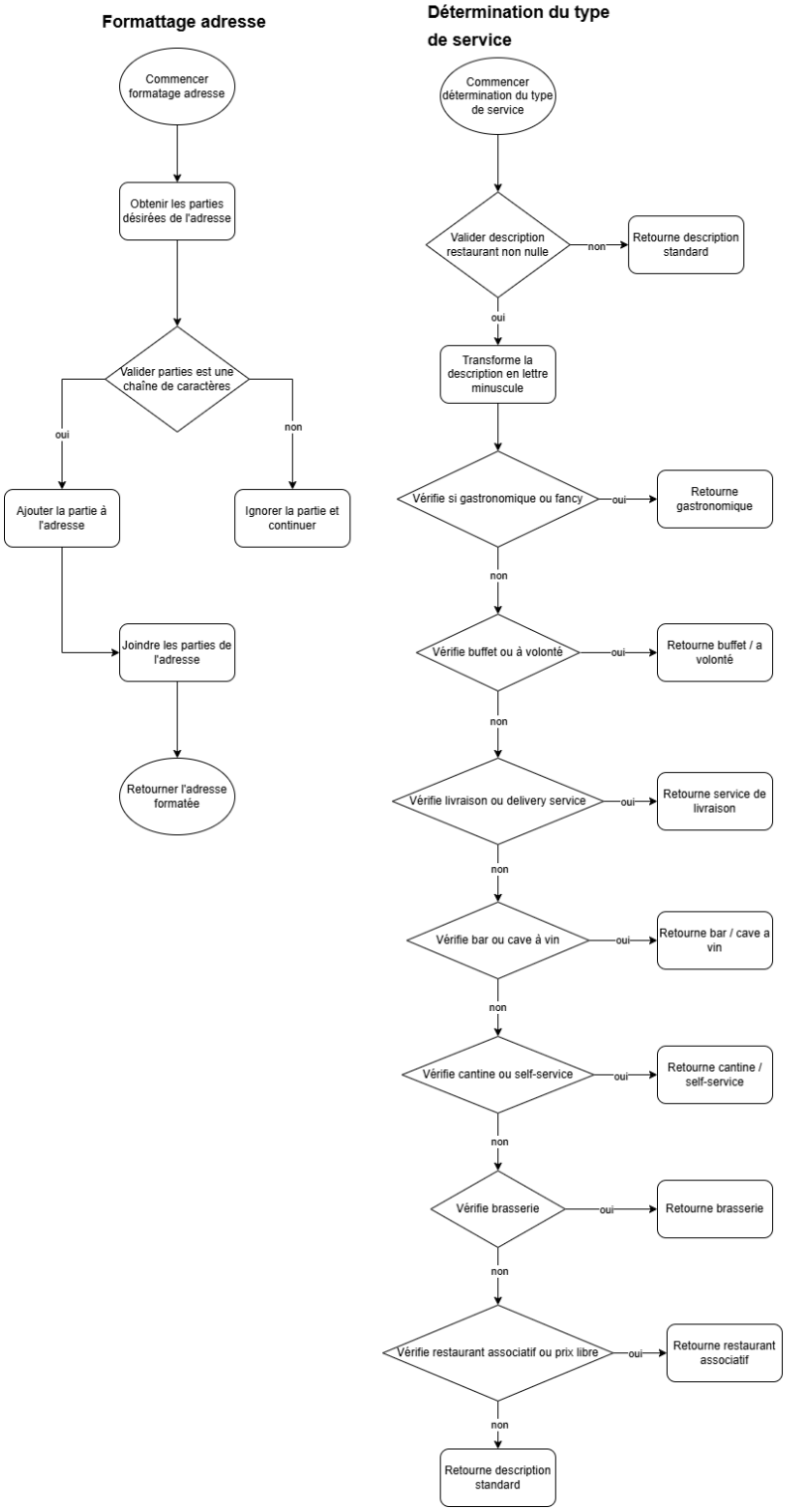
Annexe 13

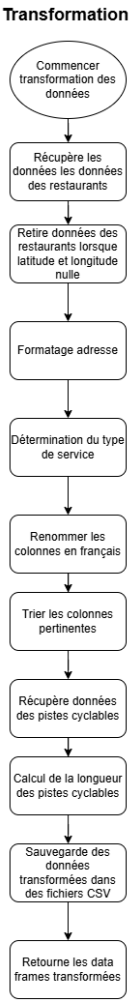
Processus d’acquisition incrémentale



Annexe 14

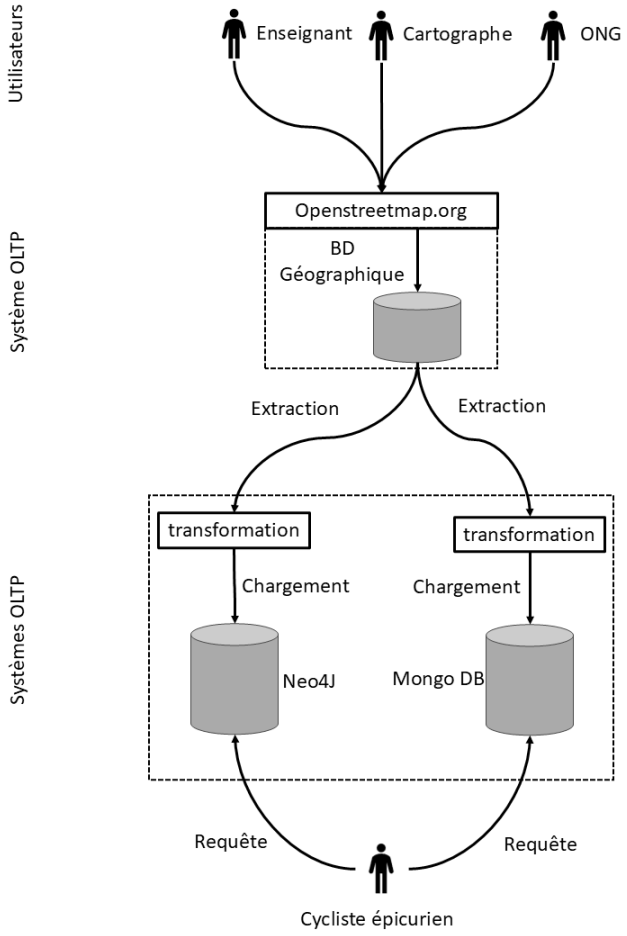
Processus de transformation





Annexe 15

Schéma du pipeline d’ETL



Annexe 16

Utilisation de l’intelligence artificielle

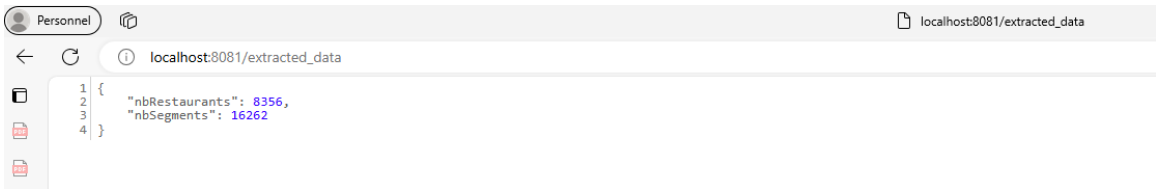
Certaines parties de ce projet ont été réalisées avec l’aide de l’assistant IA ChatGPT, en particulier pour :

1. **Script de vérification de la disponibilité de Neo4j :** Le script wait_for_neo4j() utilisé dans load.py pour s'assurer que Neo4j est prêt avant le chargement des données. Ce script attend la disponibilité de Neo4j en tentant de se connecter de manière répétée jusqu'à ce que le service soit prêt.
2. **Exécution automatique des scripts dans le Dockerfile :** La commande utilisée dans le Dockerfile pour exécuter les scripts ETL (extract.py, transform.py, load.py) a également été suggérée par ChatGPT. Cette commande garantit que le processus ETL se lance automatiquement lors du démarrage des conteneurs avec docker-compose up.
3. **Génération de diagramme UML :** Pour améliorer l’efficacité de notre création de diagramme UML, nous avons utilisé l’outil DiagramGPT. Cet outil nous a permis de générer des ébauches de diagramme basé sur notre code, que nous avons bien évidemment adapté pour qu’ils correspondent à notre travail. On a donc pu avoir plus de facilité le processus de création de diagramme UML.

Ces contributions ont permis de résoudre des problèmes techniques et d’optimiser le déploiement du projet en environnement Docker.

Annexe 17

Résultat de la requête d’extraction des données



Annexe 18

Résultat de la requête de récupération des données transformées

Personnel

localhost:8081/transformed_data

localhost:8081/transformed_data


```
{
    "longueurCyclable": 290548.95477024483,
    "restaurants": {
        "Auvergne": 1,
        "Bar / Cave à vin": 17,
        "Bar à vins": 1,
        "Brasserie": 10,
        "Buffet / À volonté": 3,
        "Cantine / Self-service": 6,
        "Cap Vert;brazilian": 1,
        "Chinese; sichuan": 1,
        "Chinese;火鍋": 1,
        "Congo": 1,
        "Gastronomique": 2,
        "Kazakh;Russe": 1,
        "New-french": 1,
        "Quebec": 1,
        "Restaurant associatif": 3,
        "Savoie": 1,
        "Service de livraison": 5,
        "Service à emporter": 4,
        "Sichuan": 1,
        "Standard": 8303,
        "Street Food": 3,
        "Tunisian": 1,
        "afghan": 4,
        "african": 54,
        "african;asian;japanese;maghreb;sushi;vietnamese": 1,
        "african;burger": 1,
        "african;burger;caribbean": 1,
        "african;couscous": 1,
        "african;ethiopian": 1,
        "african;vietnamese": 1,
        "afro-cuban;tapas": 1,
        "algerian": 2,
        "algerian;grill": 1,
        "algerian;mediterranean": 1,
        "alsatian;flammkuchen": 2,
        "american": 15,
        "american;diner": 1,
        "american;fast_food;burger": 1,
        "american;french": 1,
        "arab": 11,
        "arab;mediterranean;oriental": 1,
        "arab;moroccan": 1,
        "arepa": 1,
        "argentine": 1,
        "argentinian": 22,
        "armenian": 2,
        "asian": 264,
        "asian;barbecue;korean": 1,
        "asian;bento;korean": 1,
        "asian;bubble_tea;korean": 1,
        "asian;cambodian": 1,
        "asian;chinese": 9,
        "asian;coffee_shop;crepe;ice_cream": 1,
        "asian;french": 1,
        "asian;fusion": 1,
        "asian;japanese": 1,
        "asian;japanese;chinese": 1,
        "asian;korean;japanese": 1,
        "asian;lao": 1,
        "asian;lao;thai": 1,
        "asian;noodle": 1,
        "asian;noodle;ramen": 1,
        "asian;pasta": 1,
        "asian;ramen": 1,
        "asian;sushi": 3,
        "asian;taiwan;japan;vietnam": 1,
        "asian;thai": 2,
        "asian;thai;brunch": 1,
```